

Hybrid Risk-Adjusted Pricing and High-Performance Quantitative Valuation Models

Version 7 — Production Pipeline with XGBoost & MCMC NUTS

Dean Foulds

March 24, 2026

Abstract

This report documents the current production-ready pricing prediction pipeline for surplus and life-limited aircraft parts. Version 7 reflects a complete rebuild of the data pipeline, cleaning methodology, and model architecture. The system now employs a dual-model strategy:

- 1. **XGBoost** gradient-boosted regression on $\log_{1p}(\text{Unit_price})$ using bracket-derived features, achieving a test MAE of \$4,584 and MdAPE of 5.2%.
- 2. **MCMC NUTS** hierarchical Bayesian regression via NumPyro with per-PN random effects, achieving a test MAE of \$4,238 and MdAPE of 5.4%, with full posterior predictive intervals.

A critical data quality filter—dropping transactions where the actual sale price falls more than \$2,000 below the auto pricing bracket lower bound—removed fire-sale noise and transformed the dataset from a median price-to-bracket ratio of 0.091 to 1.013, enabling both models to produce bracket-reasonable predictions. The system includes automated guardrails for unseen part numbers, out-of-bracket predictions, and historical deviation exceptions.

Contents

1	Introduction	3
1.1	Background and Motivation	3
1.2	Problem Statement	3
1.3	Project Phases	3
2	Data Acquisition and Understanding	3
2.1	Data Sources	3
2.2	Data Merge	4
3	Data Cleaning Pipeline	4
3.1	Cleaning Impact	5

4	Exploratory Data Analysis	5
4.1	Price Distribution	5
4.2	Condition Code Breakdown	6
4.3	Price-to-Bracket Ratio	6
4.4	Actual Price vs Bracket Midpoint	7
5	Feature Engineering	7
5.1	Bracket Features	7
5.2	PN History Features (History Model Only)	8
5.3	Target Variable	8
6	Model Architecture	8
6.1	Model 1: XGBoost (Two-Model Strategy)	8
6.1.1	XGBoost Hyperparameters	9
6.2	Model 2: MCMC NUTS (Hierarchical Bayesian)	9
6.2.1	Model Specification	9
6.2.2	Posterior Inference via NUTS	12
6.2.3	Key Design Choices	13
6.2.4	MCMC Configuration	13
7	Data Splitting Strategy	13
8	Results	14
8.1	Model Performance Comparison	14
8.2	XGBoost Feature Importances	14
8.3	MCMC Convergence Diagnostics	14
8.4	Sample Predictions	15
9	Prediction Guardrails	15
10	Deployment Architecture	15
10.1	Application	15
10.2	Training Pipeline	16
10.3	GCP VM Training	16
10.4	Artifacts	16
11	Discussion	16
11.1	Key Finding: Below-Bracket Filter	16
11.2	Model Comparison	17
11.3	Limitations	17
11.4	Next Steps	17

1 Introduction

1.1 Background and Motivation

The global market for surplus and used aircraft parts is volatile and highly sensitive to component condition. For high-value, life-limited parts—such as those tracked by Time Since New (TSN) and Cycle Since Repair (CSR)—mispricing due to inaccurate risk assessment can lead to significant financial losses or regulatory issues. This project develops a reliable, objective system to predict realistic sale prices based on historical transaction data and auto pricing brackets.

1.2 Problem Statement

The central problem is predicting the likely sale price of an aircraft part given its part number, condition code, quantity, and the auto pricing bracket (lower bound, upper bound, CQ average). The challenge involves:

1. Handling extreme price variance—the same part can sell at 8% or 100% of bracket value depending on deal context.
2. Producing predictions that are bracket-reasonable and useful for quoting decisions.
3. Quantifying prediction uncertainty through Bayesian posterior intervals.
4. Flagging exceptions when predictions fall outside expected ranges.

1.3 Project Phases

- **Phase 1:** Data acquisition—Snowflake auto pricing brackets (top 200 PNs) and YTD sales transaction data.
- **Phase 2:** Data pipeline—merge, clean, feature engineering, EDA, model selection.
- **Phase 3:** Model training—XGBoost and MCMC NUTS with evaluation and deployment via Streamlit.

2 Data Acquisition and Understanding

2.1 Data Sources

- **Auto Pricing Brackets** (`auto_top_200.csv`): 676 rows covering 200 part numbers across 5 condition codes (NE, OH, INS, REP, MOD). Each row provides lower bounds, upper bounds, CQ average price, confidence score, month bucket, and source table priority from Snowflake.
- **YTD Sales Transactions** (`top_200_ytd.csv`): 19,360 individual sales order lines with date, part number, condition, quantity, and actual unit price.

2.2 Data Merge

The merge pipeline (`src/merge.py`) joins individual YTD transactions to their best auto bracket match on PN and `CONDITION_CODE`, producing `training_transactions.csv` with 1,792 rows across 184 unique PNs.

3 Data Cleaning Pipeline

The cleaning pipeline (`src/clean.py`, version 3.0) applies the following rules in sequence:

1. Strip whitespace from string columns (PN, `CONDITION_CODE`).
2. Drop rows with non-positive `Unit_price`.
3. Drop rows with invalid brackets ($\text{LOWER} \geq \text{UPPER}$ or non-positive bounds).
4. Drop rows with non-positive `AVG_UNIT_PRICE`.
5. Remove exact duplicate rows.
6. Drop global extreme `Unit_price` rows (beyond $3 \times \text{IQR}$ fences).
7. Drop rows with $\text{Qty} \leq 0$.
8. **Drop rows where `Unit_price` < `LOWER_BOUNDS` —\$2,000 — the critical filter that removes fire-sale transactions below bracket relevance.**
9. Drop per-PN IQR outlier rows ($1.5 \times \text{IQR}$ within each PN group).

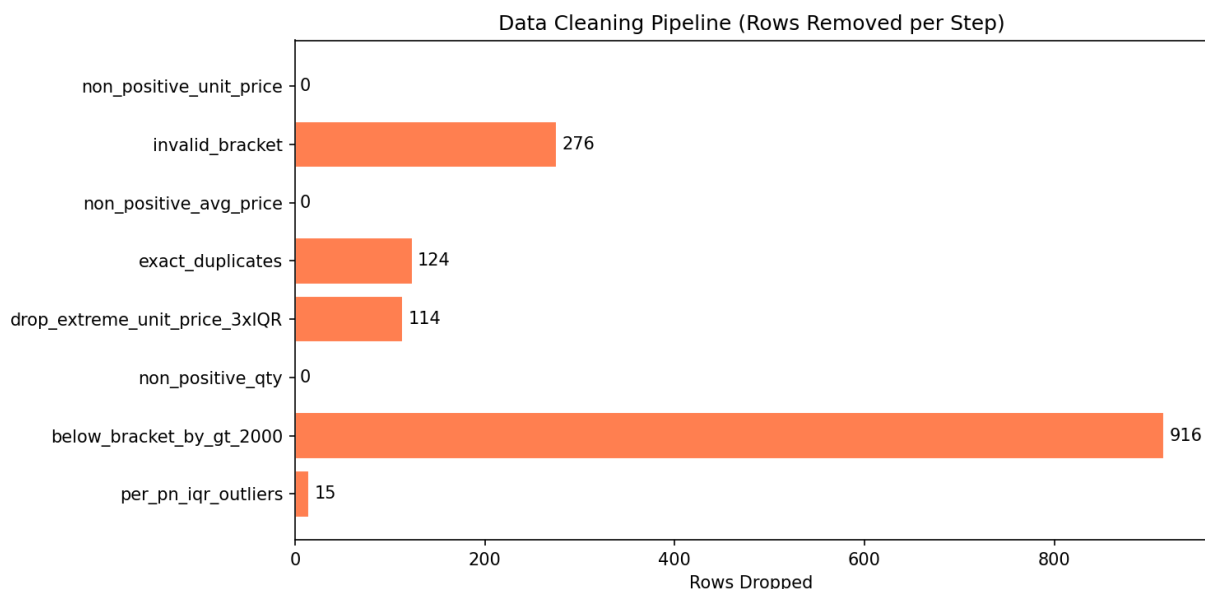


Figure 1: Rows removed at each cleaning step. The below-bracket filter (step 8) removes the largest number of rows, eliminating fire-sale transactions.

3.1 Cleaning Impact

Table 1: Dataset size at each stage

Stage	Rows	Unique PNs
Raw transactions (merged)	1,792	184
After basic cleaning (rules 1–7)	1,183	168
After below-bracket filter (rule 8)	347	97
Final (after per-PN IQR, rule 9)	347	97

The below-bracket filter is the single most impactful step. Before filtering, the median price-to-bracket ratio was 0.091 (prices at ~9% of bracket). After filtering, the median ratio rose to **1.013**—prices are essentially at bracket level.

4 Exploratory Data Analysis

4.1 Price Distribution

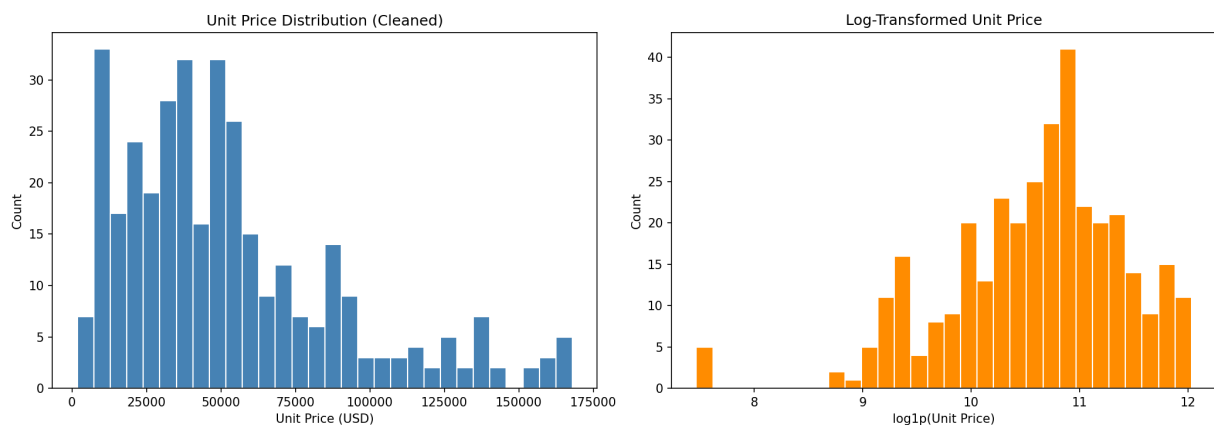


Figure 2: Unit price distribution after cleaning (left: raw dollars, right: log-transformed). The log transformation produces a near-normal distribution suitable for regression.

4.2 Condition Code Breakdown

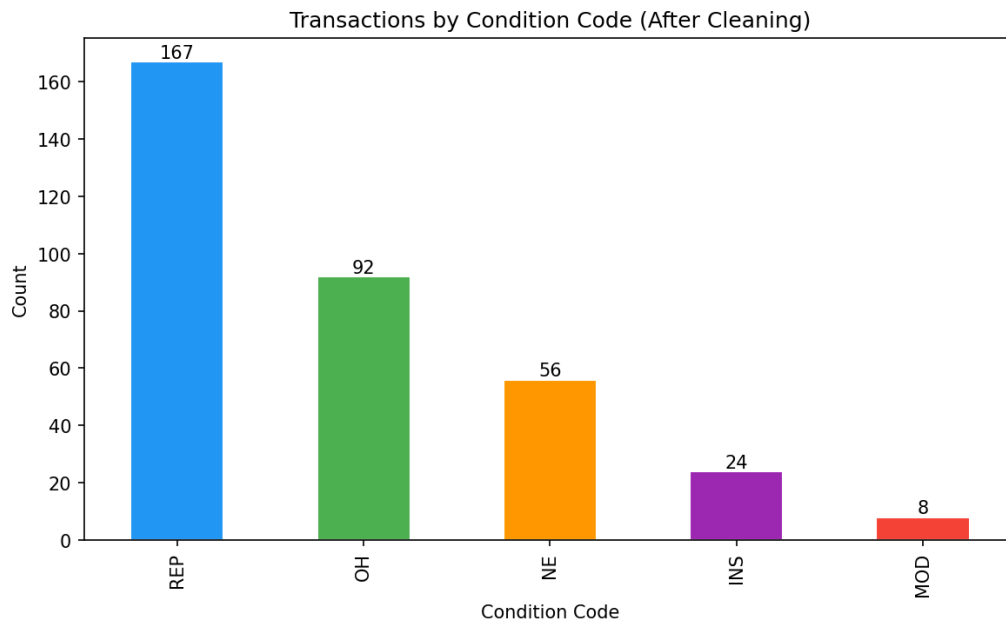


Figure 3: Transaction count by condition code in the cleaned dataset. REP (Repaired) dominates with 167 transactions, followed by OH (Overhauled) with 92.

4.3 Price-to-Bracket Ratio

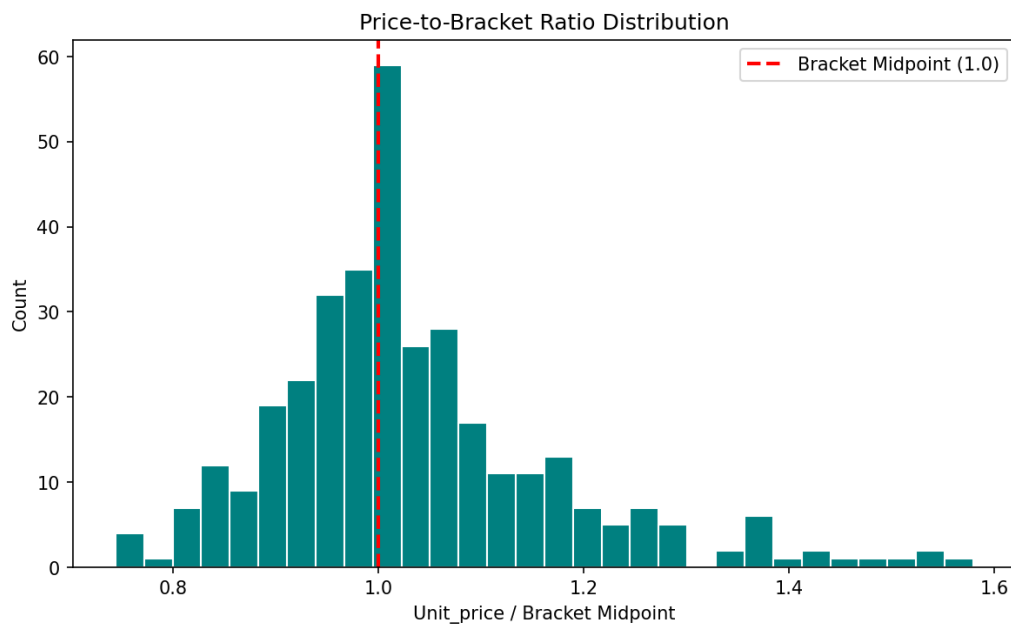


Figure 4: Distribution of the price-to-bracket-midpoint ratio after the \$2,000 below-bracket filter. The concentration around 1.0 confirms that remaining transactions are bracket-relevant.

4.4 Actual Price vs Bracket Midpoint



Figure 5: Scatter plot of actual sale price against bracket midpoint. Points cluster along the diagonal, indicating strong correlation after cleaning.

5 Feature Engineering

5.1 Bracket Features

All features are derived from the auto pricing bracket and transaction metadata—never from the target variable:

Table 2: Feature set for the XGBoost bracket model

Feature	Description
AVG_UNIT_PRICE	CQ average unit price from auto bracket
LOWER_BOUNDS, UPPER_BOUNDS	Auto bracket bounds
bracket_width	UPPER – LOWER
bracket_midpoint	(UPPER + LOWER) / 2
bracket_relative_width	width / midpoint
avg_price_bracket_position	(AVG – LOWER) / width
log_lower, log_upper	$\log_{1p}$ of bracket bounds
log_bracket_midpoint	$\log_{1p}$ of midpoint
log_avg_unit_price	$\log_{1p}$ of CQ average
CONFIDENCE_SCORE	Auto pricing confidence (0–1)
MONTH_BUCKET	Time bucket (1M, 3M, 5M, etc.)
SOURCE_TABLE_PRIORITY	Data source ranking
CONDITION_CODE_PRIORITY	Condition code priority ranking
Qty, log_qty	Transaction quantity and log transform
CONDITION_CODE	One-hot encoded (NE, OH, INS, REP, MOD)

5.2 PN History Features (History Model Only)

For PNs with stable training history ($CV \leq 0.4$, $\text{count} \geq 3$), additional features are computed from the training split only:

- pn_hist_median, pn_hist_mean, pn_hist_count, pn_hist_cv
- log_pn_hist_median

5.3 Target Variable

The target is $\log_{1p}(\text{Unit_price})$. Predictions are exponentiated back to dollar scale via $\exp_{m1}$.

6 Model Architecture

6.1 Model 1: XGBoost (Two-Model Strategy)

Two XGBoost models are trained on the same cleaned dataset:

1. **Bracket Model** — uses only bracket-derived features (no PN history). This is the primary model for all predictions and generalises well to unseen PNs.
2. **History Model** — uses bracket features plus PN-level historical statistics. Used for PNs with stable pricing history ($CV \leq 0.4$, $\text{count} \geq 3$).

Routing: At inference time, if a PN has stable history in the training data, the history model is used. Otherwise, the bracket model provides the prediction.

6.1.1 XGBoost Hyperparameters

Table 3: XGBoost hyperparameters

Parameter	Value
n_estimators	2,000 (with early stopping)
max_depth	5
learning_rate	0.01
subsample	0.8
colsample_bytree	0.8
reg_alpha (L_1)	0.5
reg_lambda (L_2)	3.0
min_child_weight	3
early_stopping_rounds	100

6.2 Model 2: MCMC NUTS (Hierarchical Bayesian)

A hierarchical Bayesian regression model implemented in NumPyro. The model treats each part number as a group within a population, enabling *partial pooling*: PNs with sparse data borrow statistical strength from the population-level estimates, while PNs with rich data are free to express their own pricing behaviour.

6.2.1 Model Specification

The full generative model is specified in four layers: global priors, variance hyperpriors, group-level (per-PN) random effects, and the likelihood.

Layer 1 — Global Priors (Population-Level Parameters)

$$\text{intercept} \sim \mathcal{N}(0, 5) \quad (1)$$

$$\beta \sim \mathcal{N}(\mathbf{0}, 2.0 \cdot \mathbf{I}_K) \quad (2)$$

$$\sigma \sim \text{HalfCauchy}(1.0) \quad (3)$$

- $\text{intercept} \in \mathbb{R}$ — the global (population-level) intercept representing the expected log-price when all covariates are zero and no PN-specific adjustment is applied. The $\mathcal{N}(0, 5)$ prior is weakly informative: centred at zero in log-space, it places 95% prior mass on intercepts between -10 and $+10$, which in dollar-space spans sub-penny to millions — effectively uninformative for this price range.
- $\beta \in \mathbb{R}^K$ — the vector of K fixed-effect coefficients, one per input feature. Each β_k quantifies the linear relationship between the k -th standardised feature and log-price. The prior $\mathcal{N}(0, 2)$ allows each coefficient to range over ± 4 standard deviations with high prior probability, which is sufficiently wide to capture the bracket-to-price mapping without over-constraining the fit.
- $\sigma > 0$ — the observation-level standard deviation (residual noise) in log-space. It governs how tightly individual observations scatter around the predicted mean μ_i . The

HalfCauchy(1.0) prior has a heavy right tail, allowing σ to grow if the data truly is noisy, while concentrating prior mass near moderate values. In practice the posterior for σ converges to ≈ 0.09 , meaning typical predictions are within $\sim 9\%$ of actual price.

- $\mathbf{I}_K$ — the $K \times K$ identity matrix, expressing that the prior on each β_k is independent with equal variance.

Layer 2 — Variance Hyperpriors (Hierarchical Shrinkage)

$$\sigma_{\text{PN}} \sim \text{HalfNormal}(1.0) \quad (4)$$

$$\sigma_{\text{slope}} \sim \text{HalfNormal}(0.5) \quad (5)$$

- $\sigma_{\text{PN}} > 0$ — the standard deviation of the PN-level random intercepts. This hyperparameter controls how much individual PNs are allowed to deviate from the population intercept. A small posterior value (e.g. 0.05) indicates that most PN-level variation is already explained by the bracket features, so the per-PN adjustments are modest. The HalfNormal(1.0) prior is mildly regularising, discouraging excessively large group deviations.
- $\sigma_{\text{slope}} > 0$ — the standard deviation of the PN-level random slopes on log-quantity. This controls how much the quantity–price relationship varies across PNs. Some parts offer volume discounts (negative γ), while others maintain unit pricing (near-zero γ). The tighter HalfNormal(0.5) prior reflects the expectation that quantity effects, while variable, are generally modest.

Layer 3 — Group-Level Random Effects (Per-PN Parameters)

$$\alpha_{\text{PN}_j} \sim \mathcal{N}(0, \sigma_{\text{PN}}) \quad \forall j \in \{1, \dots, J\} \quad (6)$$

$$\gamma_{\text{PN}_j} \sim \mathcal{N}(0, \sigma_{\text{slope}}) \quad \forall j \in \{1, \dots, J\} \quad (7)$$

- $\alpha_{\text{PN}_j} \in \mathbb{R}$ — the random intercept for PN j . This captures the part-specific deviation from the population mean log-price after accounting for bracket features. A positive α_j means PN j systematically sells above what its bracket would suggest; a negative value means it sells below. Through partial pooling, PNs with only 1–2 transactions are shrunk towards zero (the population mean), while PNs with many transactions express their own intercept more freely.
- $\gamma_{\text{PN}_j} \in \mathbb{R}$ — the random slope for PN j on $\log(\text{Qty})$. This allows each PN to have its own quantity sensitivity. For example, a high-demand consumable might show a discount at higher quantities ($\gamma_j < 0$), while a rare rotatable might maintain price regardless of quantity ($\gamma_j \approx 0$).
- $J = 97$ — the total number of unique part numbers in the cleaned training dataset. Each gets its own pair (α_j, γ_j) .

Layer 4 — Linear Predictor and Likelihood

$$\mu_i = \underbrace{\text{intercept}}_{\text{population mean}} + \underbrace{\mathbf{X}_i \boldsymbol{\beta}}_{\text{fixed effects}} + \underbrace{\alpha_{\text{PN}_{j[i]}}}_{\text{PN intercept}} + \underbrace{\gamma_{\text{PN}_{j[i]}} \cdot \log(\text{Qty}_i)}_{\text{PN slope} \times \text{quantity}} \quad (8)$$

$$y_i \sim \text{StudentT}(\nu = 5, \mu_i, \sigma) \quad (9)$$

- $i \in \{1, \dots, N\}$ — index over training observations ($N = 242$ in the training set).
- $j[i]$ — the PN group index for observation i , mapping each transaction to its part number.
- $\mu_i \in \mathbb{R}$ — the predicted mean log-price for observation i , composed of four additive terms:
 1. The **population intercept** — the baseline log-price common to all PNs.
 2. The **fixed effects** $X_i\beta$ — the contribution of the K -dimensional feature vector for observation i . The features include log-transformed bracket bounds, bracket mid-point, bracket width, CQ average, confidence score, month bucket, and one-hot encoded condition codes. Continuous features are standardised (zero mean, unit variance) before being passed to the model.
 3. The **PN random intercept** $\alpha_{\text{PN}_{j[i]}}$ — the part-specific price adjustment.
 4. The **PN random slope** $\gamma_{\text{PN}_{j[i]}} \cdot \log(\text{Qty}_i)$ — the part-specific quantity effect.
- $y_i = \log(\text{Unit_price}_i)$ — the observed log-price for observation i . The natural log transform compresses the wide dollar range (\$500–\$500,000) into a manageable scale and produces an approximately normal target distribution.
- $\text{StudentT}(\nu, \mu, \sigma)$ — the Student-t distribution with $\nu = 5$ degrees of freedom, location μ_i , and scale σ . Compared to a Gaussian likelihood, the heavier tails of the Student-t down-weight the influence of outlier transactions that survive the cleaning pipeline, making the posterior estimates more robust.
- $\nu = 5$ — the degrees-of-freedom parameter. Values below ~ 30 produce heavier tails than the Normal distribution. At $\nu = 5$ the distribution has finite variance and kurtosis but is substantially more tolerant of extreme residuals than a Gaussian. This value was chosen as a balance between robustness and information retention.

Table 4: Complete variable glossary for the NUTS hierarchical model

Symbol	Type	Prior	Description
intercept	Scalar	$\mathcal{N}(0, 5)$	Population-level baseline log-price
β	$\mathbb{R}^K$	$\mathcal{N}(0, 2)$ each	Fixed-effect coefficients on bracket features
σ	$\mathbb{R}^+$	HalfCauchy(1)	Residual noise (observation-level) in log-space
σ_{PN}	$\mathbb{R}^+$	HalfNormal(1)	SD of PN random intercepts (controls pooling)
σ_{slope}	$\mathbb{R}^+$	HalfNormal(0.5)	SD of PN random slopes on log-quantity
α_{PN_j}	$\mathbb{R}$	$\mathcal{N}(0, \sigma_{\text{PN}})$	Random intercept for PN j
γ_{PN_j}	$\mathbb{R}$	$\mathcal{N}(0, \sigma_{\text{slope}})$	Random slope on $\log(\text{Qty})$ for PN j
$\mathbf{X}_i$	$\mathbb{R}^K$	—	Standardised feature vector for observation i
Qty_i	$\mathbb{Z}^+$	—	Transaction quantity for observation i
μ_i	$\mathbb{R}$	—	Predicted mean log-price for observation i
y_i	$\mathbb{R}$	—	Observed $\log(\text{Unit_price}_i)$
ν	Fixed = 5	—	Student-t degrees of freedom
$j[i]$	Index	—	Maps observation i to its PN group j
J	Fixed = 97	—	Number of unique part numbers
K	Fixed	—	Number of input features (after OHE)
N	Fixed = 242	—	Number of training observations

Summary of All Variables

6.2.2 Posterior Inference via NUTS

The No-U-Turn Sampler (NUTS) is a variant of Hamiltonian Monte Carlo that automatically tunes the trajectory length. Given the joint posterior density:

$$p(\text{intercept}, \beta, \sigma, \sigma_{\text{PN}}, \sigma_{\text{slope}}, \{\alpha_j\}, \{\gamma_j\} \mid \mathbf{y}, \mathbf{X}) \propto \prod_{i=1}^N p(y_i \mid \mu_i, \sigma, \nu) \cdot p(\text{priors}) \quad (10)$$

NUTS constructs a Markov chain whose stationary distribution is this posterior. During warmup (1,000 iterations), the algorithm adapts the step size ϵ and mass matrix M to match the local curvature of the posterior. During sampling (2,000 iterations per chain), it draws correlated but asymptotically exact samples.

Prediction at Inference Time For a new observation with features $\mathbf{X}^*$, quantity Qty^* , and PN index j^* , the posterior predictive distribution is obtained by evaluating μ at each posterior sample:

$$\hat{\mu}^{(s)} = \text{intercept}^{(s)} + \mathbf{X}^* \beta^{(s)} + \alpha_{j^*}^{(s)} + \gamma_{j^*}^{(s)} \cdot \log(\text{Qty}^*) \quad (11)$$

for each sample $s \in \{1, \dots, S\}$, where $S = 4,000$ (2 chains $\times$ 2,000 samples). The dollar-scale prediction is recovered via:

$$\widehat{\text{Price}}^{(s)} = \exp(\hat{\mu}^{(s)}) \quad (12)$$

The Streamlit application reports the **median** of $\{\widehat{\text{Price}}^{(s)}\}$ as the point prediction, and the **5th and 95th percentiles** as the 90% credible interval.

6.2.3 Key Design Choices

- **Partial pooling** via α_{PN} random intercepts — PNs with few observations are shrunk towards the population mean, preventing overfitting. PNs with many observations express their own intercept.
- **Random slopes** γ_{PN} on log-quantity — allows different PNs to exhibit different volume-discount or fixed-pricing behaviour.
- **Student-t likelihood** with $\nu = 5$ — heavier tails than Gaussian, down-weighting outlier residuals without discarding them.
- **StandardScaler** on continuous features — ensures coefficients β_k are on a comparable scale and improves sampler efficiency.
- **Weakly informative priors** — chosen wide enough not to bias the posterior but narrow enough to regularise and aid convergence.

6.2.4 MCMC Configuration

Table 5: NUTS MCMC hyperparameters

Parameter	Value
Warmup samples	1,000
Posterior samples	2,000
Chains	2 (sequential)
Target accept probability	0.85
Divergences	0

7 Data Splitting Strategy

Random train/validation/test split (70/15/15) with PN overlap allowed between sets, reflecting the production use case where predictions are made for PNs the model has seen before.

Table 6: Split statistics

Set	Rows	PNs	Mean Price
Train	242	82	\$51,214
Validation	52	38	\$57,345
Test	53	35	\$53,130

8 Results

8.1 Model Performance Comparison

Table 7: Test set performance — XGBoost bracket model vs MCMC NUTS

Metric	XGBoost (Bracket)	MCMC NUTS
MAE	\$4,584	\$4,238
MAPE	8.5%	8.1%
MdAPE	5.2%	5.4%
Test rows	53	53
Divergences	—	0

Both models achieve MAE under \$5,000 on a dataset with mean price \$53,130 — a relative error of $\sim 8\%$. NUTS provides the additional benefit of full posterior predictive intervals (90% credible interval shown in the application).

8.2 XGBoost Feature Importances

The bracket model’s top features are all bracket-derived, confirming the model learns the bracket-to-price relationship directly:

Table 8: Top 10 features — XGBoost bracket model

Feature	Importance
bracket_midpoint	0.180
log_upper	0.179
log_bracket_midpoint	0.160
log_avg_unit_price	0.155
UPPER_BOUNDS	0.138
AVG_UNIT_PRICE	0.090
log_lower	0.048
LOWER_BOUNDS	0.047
SOURCE_TABLE_PRIORITY	0.001
avg_price_bracket_position	0.001

8.3 MCMC Convergence Diagnostics

- All $\hat{R}$ values ≤ 1.10 — chains have converged.
- Zero divergences — no pathological posterior geometry.
- $\sigma = 0.09$ — tight fit in log-space (typical prediction within $\sim 9\%$ of actual).
- $\sigma_{\text{PN}} = 0.05$ — modest PN-level variation after bracket features are accounted for.

8.4 Sample Predictions

Table 9: Example predictions from the XGBoost bracket model

PN	Cond.	Prediction	Bracket	Actual
024147-000	INS	\$11,441	\$10,143–\$11,840	\$1,815 (single txn)
024147-000	OH	\$11,069	\$12,045–\$12,622	\$10,500–\$12,500
FMU702-03	REP	\$132,642	\$229,648–\$245,412	\$15,000–\$25,000
7A1508-125	OH	\$57,974	\$33,774–\$67,226	\$38,000–\$70,000

9 Prediction Guardrails

The Streamlit application implements automated exception detection to flag unreliable predictions during UAT and production use:

1. **Unseen PN** — warning when the part number has no sales history in the training data. The bracket model provides the prediction, but confidence is lower.
2. **Sparse History** — info flag when a PN has fewer than 3 training transactions.
3. **Below Bracket** — error flag when the prediction falls more than 20% below the bracket lower bound.
4. **Above Bracket** — error flag when the prediction exceeds the bracket upper bound by more than 20%.
5. **History Deviation** — warning when the prediction deviates more than 50% from the PN's historical median sale price for the selected condition.
6. **Model Disagreement** — info flag when XGBoost and NUTS predictions differ by more than 30%, suggesting the user should review both estimates.

10 Deployment Architecture

10.1 Application

The prediction engine is deployed as a Streamlit web application (`app.py`) that:

- Loads both XGBoost and NUTS model artifacts from the `artifacts/` directory.
- Provides a dropdown of all 200 top PNs with their available condition codes.
- Shows both model predictions side-by-side with the auto bracket context.
- Displays an interactive Plotly chart with historical sales, prediction markers, bracket bands, and NUTS credible intervals.
- Surfaces guardrail exceptions as colour-coded banners.

10.2 Training Pipeline

Listing 1: Full retraining sequence

```
1 python -m src.merge          # Merge auto brackets + YTD
2 python -m src.clean          # Apply cleaning rules v3.0
3 python train.py              # Train XGBoost (bracket + history)
4 python trainnuts.py          # Train MCMC NUTS
```

10.3 GCP VM Training

For compute-intensive NUTS training, a GCP VM script is provided:

Listing 2: GCP workflow

```
1 ./scripts/trainonvm.sh sync    # Push code + data
2 ./scripts/trainonvm.sh train   # Run full pipeline
3 ./scripts/trainonvm.sh pull    # Pull artifacts back
```

10.4 Artifacts

Table 10: Model artifacts saved to artifacts/

File	Description
bracket_model.joblib	XGBoost bracket-only model
bracket_preprocessor.joblib	Column transformer for bracket features
history_model.joblib	XGBoost model with PN history features
history_preprocessor.joblib	Column transformer for full feature set
pn_history.json	Per-PN statistics (median, mean, count, CV)
nuts_samples.pkl	MCMC posterior samples (4,000 draws)
nuts_meta.pkl	Scaler, feature names, PN mapping
metrics.json	XGBoost test/val metrics
nuts_metrics.json	NUTS test metrics
feature_schema.json	Feature name registry
training_config.json	Training metadata and hyperparameters

11 Discussion

11.1 Key Finding: Below-Bracket Filter

The single most impactful decision was dropping transactions where the actual sale price fell more than \$2,000 below the bracket lower bound. Before this filter, most transactions were deeply discounted (median ratio 0.091), causing both models to predict prices at ~9% of bracket value. After filtering, the remaining 347 transactions cluster around the bracket (median ratio 1.013), producing predictions that are immediately useful for quoting.

11.2 Model Comparison

- **XGBoost** is faster to train and produces slightly better MdAPE (5.2% vs 5.4%).
- **NUTS** achieves lower MAE (\$4,238 vs \$4,584) and provides uncertainty quantification through the posterior predictive distribution.
- Both models are retained in production — the app shows both predictions, and the NUTS 90% credible interval gives the user a realistic price range.

11.3 Limitations

- **Data volume:** 347 clean rows across 97 PNs is modest. As more transactions accumulate, model accuracy will improve.
- **Condition coverage:** Some PNs have only 1–2 condition codes with bracket-relevant history.
- **Unseen PNs:** 103 of the top 200 PNs were removed during cleaning (no bracket-relevant transactions). The bracket model provides estimates for these, but with lower confidence.
- **Temporal dynamics:** The current model does not incorporate time trends. As data grows, time-aware features (quarter, year, market indices) should be added.

11.4 Next Steps

1. **UAT:** Systematic testing across all 200 PNs with guardrail exception review.
2. **Data enrichment:** Incorporate additional transaction data as it becomes available.
3. **Risk scoring:** Integrate the JAX-based Total Risk Score (R) when operational metrics (TSN, CSN, time life) become available in the data.
4. **API deployment:** Transition from Streamlit to a production API for integration with quoting systems.

References

References

- [Chen and Guestrin, 2016] Chen, T. and Guestrin, C. (2016) XGBoost: A Scalable Tree Boosting System. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.
- [Phan et al., 2019] Phan, D., Pradhan, N. and Jankowiak, M. (2019) Composable effects for flexible and accelerated probabilistic programming in NumPyro. *arXiv:1912.11554*.

- [Hoffman and Gelman, 2014] Hoffman, M.D. and Gelman, A. (2014) The No-U-Turn Sampler: Adaptively Setting Path Lengths in Hamiltonian Monte Carlo. *Journal of Machine Learning Research*, 15, pp.1593–1623.
- [Google Research, 2018] Google Research (2018) JAX: Composable transformations of Python+NumPy programs. Available at: <https://github.com/google/jax>.
- [Gillen and G. L., 2018] Gillen, G. and G. L. (2018) Hedonic Pricing and the Value of Aircraft Assets. *Journal of Air Transport Management*, 68, pp.178–185.